BentoML (adaptive batching): Locust load test results

Arabic sentiment service after moving to BentoML with `batchable=True`. PyTorch FP32 on an RTX 4050 (WSL), single worker, same locustfile as the FastAPI baseline (random real reviews from the test split, 0.5 to 2.0 s think time, stats reset after ramp-up). Numbers are the **POST /predict** row. Data extracted from three Locust HTML reports.

BentoML results

Users	Run start (UTC)	Duration	/predict requests	Fails	p50	p95	p99	Avg latency	Avg RPS /predict (all)
50	2026-10-26 21:18:30	3 minutes and 7 seconds	6,243	0	55 ms	100 ms	200 ms	61 ms	33.2 (36.6)
100	2026-10-27 22:20:32	58 seconds	3,702	0	13 ms	39 ms	110 ms	18 ms	63.7 (70.6)
1000	2026-10-27 22:03:47	2 minutes and 9 seconds	42,094	30,032 (71%)	1500 ms	1900 ms	2500 ms	1504 ms	324.1 (356.7)

FastAPI vs BentoML at the same load

Users	Service	Fails	p50	p95	p99	Successful predictions per second
50	FastAPI	0	180 ms	530 ms	1100 ms	28.8
	BentoML	0	55 ms	100 ms	200 ms	33.2
100	FastAPI	0	1400 ms	1900 ms	4200 ms	32.3
	BentoML	0	13 ms	39 ms	110 ms	63.7
1000	FastAPI	0	26.0 s	29.0 s	29.0 s	32.6
	BentoML	71%	1500 ms	1900 ms	2500 ms	92.9

What the runs show

1. No failures at 50 and 100 users. At 100 users BentoML kept p95 at 39 ms against 1900 ms for FastAPI (49x lower), and served 63.7 predict RPS against 32.3 (2.0x).
2. Capacity is about 3x higher. FastAPI plateaus near 32 predictions per second. Even at 1000 users, BentoML completed about 93 successful predictions per second while rejecting the rest.
3. At 1000 users BentoML rejects instead of queueing. 30,032 of 42,094 requests (71%) failed, all with HTTP 503 from BentoML's batching dispatcher (it cancels requests whose estimated queue time exceeds `max_latency_ms`). FastAPI instead queues everyone (p50 26 s at 1000 users, 0 failures). The 1.5 s median above is dominated by fast rejections and says nothing about the latency of successful requests.
4. A result to double-check: p50 at 100 users (13 ms) is lower than at 50 users (55 ms). The 50-user run was recorded a day earlier (26 Oct) and may have run under different conditions (for example the temporary batch-size print line in the service, or the GPU clock state). Re-running the 50-user level in the same session would settle it.

Caveats

The 100-user run lasted only 58 s and the 1000-user run 129 s (the 50-user run 3 min 7 s). One run per level. Locust and the service share the same laptop CPU, which matters most at 1000 users. Request format differs (FastAPI sends one text, BentoML a one-item list), but every request carries exactly one review. The cause of the improvement (batching) is a hypothesis: batch sizes were not recorded.

Throughput and latency over time

Left: requests per second (blue) and failures per second (red dashed). Right: p50 and p95 latency. Time starts at the stats Reset.











